

Containers e Virtualização no Mercado Moderno de TI

A evolução da infraestrutura: de máquinas virtuais pesadas a orquestradores em escala planetária.



LINHA DO TEMPO: EVOLUÇÃO DO USO DE SERVIDORES PARA APLICAÇÕES DE SOFTWARE

De Hardware Físico ao Serverless e Nuvem

1. ERA DOS SERVIDORES FÍSICOS



2. ERA DA VIRTUALIZAÇÃO



3. ERA DOS CONTAINERS



4. ERA DA NUVEM & CLOUD NATIVE



5. FUTURO: SERVERLESS & COMPUTING EDGE



1990

2000

2013

2018

2024+

SERVIDORES FÍSICOS DEDICADOS

- Hardware Único por Aplicação
- Lento provisionamento, Baixa eficiência, Manutenção manual

VIRTUALIZAÇÃO (VMs)

- Múltiplos SOs no mesmo Hardware
- Hypervisor (VMware, Xen)
- Melhor eficiência, Consolidação de Servidores

CONTAINERS & MICROSERVIÇOS

- Isolamento a nível de SO
- Docker, Kubernetes (K8s)
- Rápida inicialização, Portabilidade, Escalabilidade ágil

CLOUD NATIVE & MICROSERVIÇOS

- Infraestrutura como Código (IaC)
- Orquestração (EKS, AKS), AWS, Azure, GCP
- Escalabilidade global, Flexibilidade

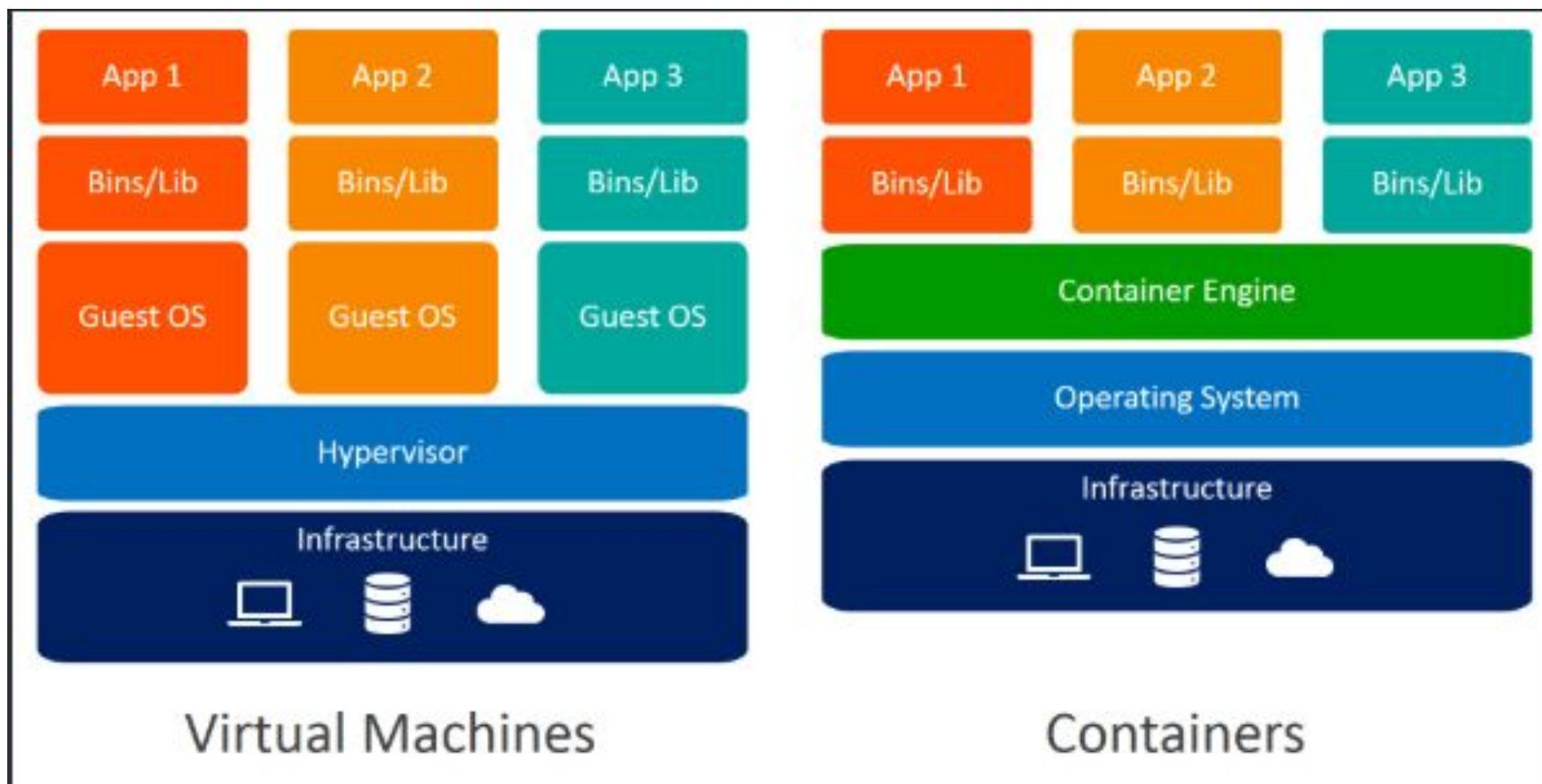
SERVERLESS & COMPUTING EDGE

- Foco no Código (FaaS) AWS Lambda, Azure Functions, Google Cloud Functions
- Pagamento por uso, Sem gerenciamento de Servidor, Alta agilidade

Máquinas Virtuais vs Containers

Comparativo técnico de isolamento e eficiência

Aspecto	Máquinas Virtuais (VM)	Containers (Docker / OCI)
Isolamento	Hardware virtualizado via Hypervisor	Nível de SO (Namespaces e cgroups)
Sistema Operacional	Guest OS completo com kernel próprio	Compartilha o mesmo kernel do Host
Tempo de Inicialização	Minutos (boot completo de SO)	Milissegundos a segundos
Tamanho em Disco	Gigabytes por instância	Megabytes em camadas UnionFS



A VM Morreu? Não, Mudou de Papel

10x – 50x

Maior densidade de aplicações por nó físico

As VMs deixaram de empacotar aplicações individuais para se tornarem o substrato de hardware virtualizado sobre o qual os clusters de containers operam.

O Dilema da Máquina do Desenvolvedor

A origem da crise de confiabilidade antes dos containers



Incompatibilidade de Ambiente

Versões divergentes de bibliotecas e SO.



Setup Local Excessivo

Dias para configurar bancos e runtimes.



Desperdício de Recursos

Uma VM completa para cada serviço.

Docker no Cotidiano do Desenvolvedor

Padronização de runtime do notebook até o servidor

Bancos e filas sob demanda: Sem instalação local de Postgres ou Redis.

Isolamento de versões: Node 16 e Node 20 sem conflitos de PATH.

Onboarding instantâneo: Clonar o repositório e executar com um único comando.



A Tríade Fundamental do Docker

Da declaração textual à instância em execução

1

Dockerfile

Receituário declarativo com comandos de montagem.

2

Imagem Docker

Snapshot binário imutável organizado em camadas.

3

Container Ativo

Instância viva com camada de escrita volátil.

Docker Compose: Ambientes Multisserviço

Coordenação declarativa de serviços, volumes e redes



Arquivo Declarativo

Todo o stack em formato YAML legível.



Redes Automáticas

Resolução DNS integrada entre serviços.



Persistência Segura

Volumes mapeados sem poluir o SO host.

Containers em Escala de Produção

Ganhos operacionais mensuráveis em infraestrutura corporativa

100%

Imutabilidade

Mesma imagem no teste e em produção.

< 5s

Autoscale Horizontal

Novas instâncias sob pico de tráfego.

75%

Uso de CPU/RAM

Redução drástica de servidores ociosos.

O Desafio dos Centenas de Containers

“

“Rodar um container na sua máquina é trivial. Manter mil containers saudáveis, seguros e comunicáveis em múltiplos datacenters exige orquestração.”

Necessidade crítica: auto-recuperação, roteamento, upgrades sem indisponibilidade e balanceamento.

Docker Swarm vs Kubernetes (K8s)

Simplicidade pragmática versus domínio corporativo absoluto

Docker Swarm

Nativo: Integrado diretamente no Docker Engine.

Curva suave: Usa sintaxe idêntica ao Compose.

Cenário ideal: Times enxutos e borda (Edge).

Kubernetes (K8s)

Padrão corporativo: Apoiado por toda a nuvem (CNCF).

Poder total: Alta extensibilidade e operadores.

Demanda: Principal requisito para vagas Cloud/DevOps.

O Que o Mercado Exige de quem está começando?

01

Dockerfiles Eficientes

Multi-stage build e redução de imagem.

02

Docker Compose Local

Subir a aplicação conectada ao banco de dados.

03

Fundamentos de SO

Namespaces, redes em ponte e variáveis de ambiente.

ENCERRAMENTO

Perguntas & Discussão Pedagógica
